
Payments & Paytm: Dual Payment Architecture & Three-Way Automated Revenue Split

A comprehensive, plain-English architectural guide to how payments work in V FOODS, how Paytm payment confirmation operates end-to-end, and how automated three-way revenue splitting (Shop, Platform, College) is handled.

1. The Dual Payment Choice: Wallet Top-Up vs. Direct Order Payment

V FOODS supports **both** payment paths for maximum speed and customer flexibility:

1. Path 1: Wallet Top-Up (Prepaid Campus Wallet)

2. **How it works:** Students add money to their digital campus wallet when convenient (e.g. from their dorm room or in the morning) via Paytm UPI or Cards.
3. **Splitting Rule: NEVER split.** A top-up is a 100% store of value credited directly to the student's wallet balance.
4. **Confirmation:** Confirmed strictly via Paytm's signed server-to-server webhook, credited through the idempotent `credit_wallet()` procedure.

5. Path 2: Direct Order Payment (Instant Gateway Checkout)

6. **How it works:** Students pay Paytm directly for one specific food order, bypassing the wallet balance completely.
7. **Splitting Rule:** Confirmed via Paytm webhook before the order is marked `placed` and sent to the kitchen KDS screen. Once confirmed, it triggers the three-way split.
8. **Why use it:** Perfect for visitors, faculty, or students who prefer direct bank debit per meal without maintaining a wallet balance.

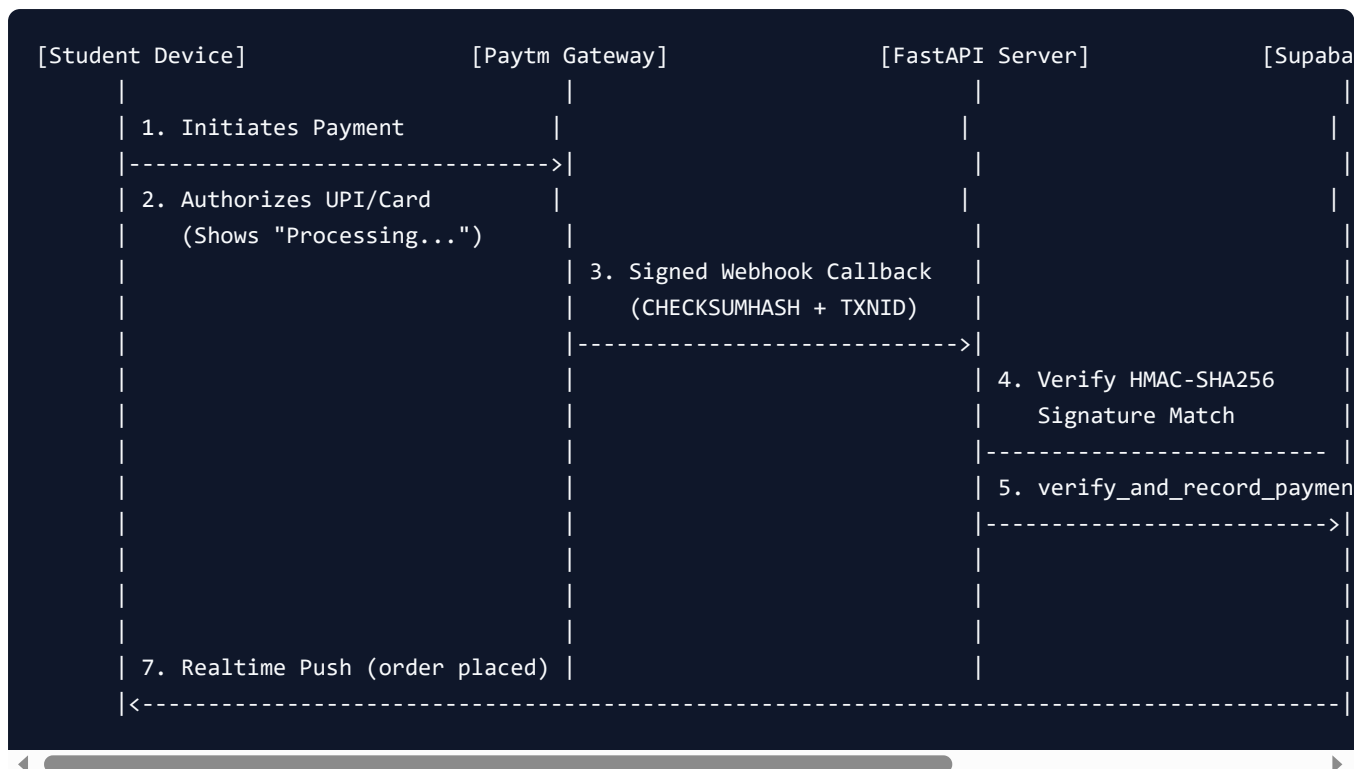
Both paths support payment methods: `UPI_ID`, `UPI_APP`, `UPI_QR`, `DEBIT_CARD`, `CREDIT_CARD` (and `WALLET` internally for campus wallet orders).

2. Confirmation of What Existed vs. What Was Added

Payment Path / Component	Status Prior to Architecture Update	Current Status After Migration 011
Wallet Top-Up Flow	Existed via PhonePe sandbox routes (<code>POST /wallet/topup</code>) and Supabase <code>credit_wallet()</code> .	Migrated to Paytm: Implemented <code>POST /wallet/topup/paytm</code> , signed payload generation, and verified webhook crediting.
Direct Order Payment Flow	Existed conceptually via <code>orders.payment_method = 'gateway'</code> , <code>create_pending_gateway_order()</code> , and <code>finalize_gateway_order()</code> .	Fully Unified: Added <code>POST /order/checkout/paytm-session</code> , <code>payments</code> record linkage, and automatic three-way split upon webhook confirmation.
Payment Gateway	PhonePe SDK mock / sandbox (<code>phonepe_txn_id</code> primary key).	Paytm Production Architecture: Replaced with Paytm merchant configuration, HMAC-SHA256 checksum verification, and <code>paytm_txn_id</code> tracking.
Revenue Splitting	Did not exist (orders only had a generated column <code>my_profit = shop_payout * 5%</code>).	Three-Way Automated Split (<code>payment_splits</code>): Configurable outlet accounts, fixed platform account, active college account, and exact penny split records.
Sub-Account Onboarding	None.	Added <code>outlet_paytm_accounts</code> , <code>platform_settlement_account</code> , and <code>college_settlement_accounts</code> .
Refunds Schema	None.	Added unified <code>refunds</code> table supporting <code>ORDER_REFUND</code> , <code>WALLET_TOPUP_REFUND</code> , and <code>SPLIT_REFUND</code> .

3. General Payment-Confirmation Flow

Payment confirmation follows strict zero-trust, server-authoritative principles:



Core Security Rules:

- 1. The frontend NEVER decides a payment succeeded:** The frontend only ever displays "Processing" until the database status updates. Even if a student's phone claims a payment finished, the backend and kitchen screens ignore it completely until the server-side callback arrives.
- 2. Paytm sends a signed webhook callback:** When the transaction completes on Paytm's banking infrastructure, Paytm's server fires an HTTPS POST callback directly to our FastAPI endpoint `/webhooks/paytm`.
- 3. Backend verifies Paytm's checksum signature:** The backend recalculates the HMAC-SHA256 checksum using the private `PAYTM_MERCHANT_KEY`. If the signature is missing, forged, or fails verification, the webhook is rejected outright with an HTTP 400 Bad Request error.
- 4. Action only after signature verification:** Only after cryptographic verification does the backend execute `verify_and_record_payment()` in PostgreSQL.
- For `WALLET_TOPUP`: Credits the wallet via the existing idempotent `credit_wallet()` stored procedure.
- For `ORDER`: Marks the order `placed`, issues the pickup token, decrements menu item stock, and triggers `calculate_order_split()`.
- 7. Idempotency against network retries:** Paytm retries webhooks multiple times if network packets drop. The system checks `payments.paytm_txn_id` and `wallet_txns.ref`. Whether the webhook arrives once or five times, it produces the exact same result with zero duplicate payments and zero double-credits.
- 8. Student connection resilience:** If the student's phone battery dies, closes the browser, or loses cellular connectivity right after paying in their UPI app, confirmation is completely unaffected. The confirmation travels server-to-server between Paytm and V Foods, not through the student's device.

4. The Three-Way Split (Shop, Platform, College)

Every meal order in V Foods—whether paid directly via Paytm or funded using the campus wallet—generates an automated, three-way revenue distribution record in `payment_splits`. Top-ups are never split.

The Three Recipients:

1. SHOP (The Fulfilling Outlet):

- Differs per shop: Each outlet has its own distinct Paytm sub-account (`outlet_paytm_accounts`) and its own configurable `shop_split_percentage` (default **90%**).
- For example, Main Canteen may have 90%, while Tech Café may have 85% or 92% based on their commercial agreement.

4. PLATFORM (V Foods Startup):

- One fixed account (`platform_settlement_account`) app-wide for the startup company (`platform_split_percentage` = **5%**).

6. COLLEGE (Host Institution / Campus):

- One fixed account (`college_settlement_accounts`) for the current active campus (`college_split_percentage` = **5%**).
- Structured with `is_active` boolean and partial unique index, allowing seamless expansion to future campuses (e.g. VIT Vellore, VIT AP, or SRM).

Percentage Balancing Rule (100% Invariant):

- The database enforces an unchangeable constraint via trigger: $\text{Shop Percentage} + \text{Platform Percentage} + \text{College Percentage} = 100\%$
 - When calculating penny split amounts from the order total, the platform and shop amounts are rounded to 2 decimal places, and the college receives the exact penny residual. This guarantees that: $\text{Split}_{\text{Shop}} + \text{Split}_{\text{Platform}} + \text{Split}_{\text{College}} \equiv \text{Order Total}$
-

5. Sub-Account Onboarding & Settlement Schema

Every recipient must be onboarded with Paytm before split payouts can be disbursed:

```

-- Outlet sub-accounts (per shop)
CREATE TABLE outlet_paytm_accounts (
  outlet_id          text PRIMARY KEY REFERENCES outlets(id),
  paytm_account_id   text NOT NULL,
  onboarding_status  text NOT NULL CHECK (onboarding_status IN ('PENDING', 'ACTIVE', 'REJECTED'),
  settlement_enabled boolean NOT NULL DEFAULT false,
  shop_split_percentage numeric NOT NULL DEFAULT 90 CHECK (shop_split_percentage BETWEEN 0 AND 100),
  created_at         timestampz DEFAULT now(),
  updated_at         timestampz DEFAULT now()
);

-- Platform settlement account (singleton startup row)
CREATE TABLE platform_settlement_account (
  id                int PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  paytm_account_id  text NOT NULL,
  settlement_enabled boolean NOT NULL DEFAULT false,
  platform_split_percentage numeric NOT NULL DEFAULT 5 CHECK (platform_split_percentage BETWEEN 0 AND 100),
  created_at        timestampz DEFAULT now(),
  updated_at        timestampz DEFAULT now()
);

-- College settlement accounts (one active campus)
CREATE TABLE college_settlement_accounts (
  id                bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  campus_name       text NOT NULL,
  paytm_account_id  text NOT NULL,
  onboarding_status text NOT NULL CHECK (onboarding_status IN ('PENDING', 'ACTIVE', 'REJECTED'),
  settlement_enabled boolean NOT NULL DEFAULT false,
  is_active         boolean NOT NULL DEFAULT true,
  college_split_percentage numeric NOT NULL DEFAULT 5 CHECK (college_split_percentage BETWEEN 0 AND 100),
  created_at        timestampz DEFAULT now(),
  updated_at        timestampz DEFAULT now()
);
CREATE UNIQUE INDEX idx_college_single_active ON college_settlement_accounts(is_active) WHERE is_active = true;

```

6. Security Confirmation & PCI Compliance

- **No Sensitive Financial Credentials Stored:** No column anywhere in the V Foods database stores credit/debit card numbers, CVVs, expiry dates, or UPI PINs.
- **Paytm Secret Isolation:** Paytm merchant secret keys (`PAYTM_MERCHANT_KEY`) are stored solely as environment variables in the secure FastAPI server runtime and never exposed to the frontend or database tables.
- **Row Level Security (RLS):**
 - Students can only view their own payment and refund records.
 - Shop Staff and Shop Admins can only view payments and `SHOP` -type split rows for their own outlet.
 - Platform and College settlement accounts, and platform/college split records, are restricted exclusively to Super Admins.

